

IEEE 1687 (IJTAG)

波形解析與逐拍追蹤指南

從 TAP state、CE/SE/UE、SIB 到 TDI/TDO/TDO_EN 的完整驗證流程

適用情境 Tessent 自動產生的 IJTAG pattern、RTL/gate-level simulation waveform、ICL network connectivity 驗證，以及報告時需要說明「為什麼這個波形是對的」。

版本：2026-07-29

文件語言：繁體中文 | 技術名詞保留英文

範圍：IEEE 1687 + 1149.1 hosting；IEEE 1838 僅在必要處說明界線

閱讀導引與最重要結論

這份文件的目的是不是只解釋訊號名稱，而是提供一套可以逐拍證明 pattern 正確與否的方法。面對 CE、SE、UE、TDI、TDO、SIB、TDE 與 TDO_EN 混在同一張 waveform 的情況，最容易出錯的做法是把所有訊號當成同一層級、用視覺上的高低重疊直接下結論。正確方法是分層重建協定、路徑與資料。

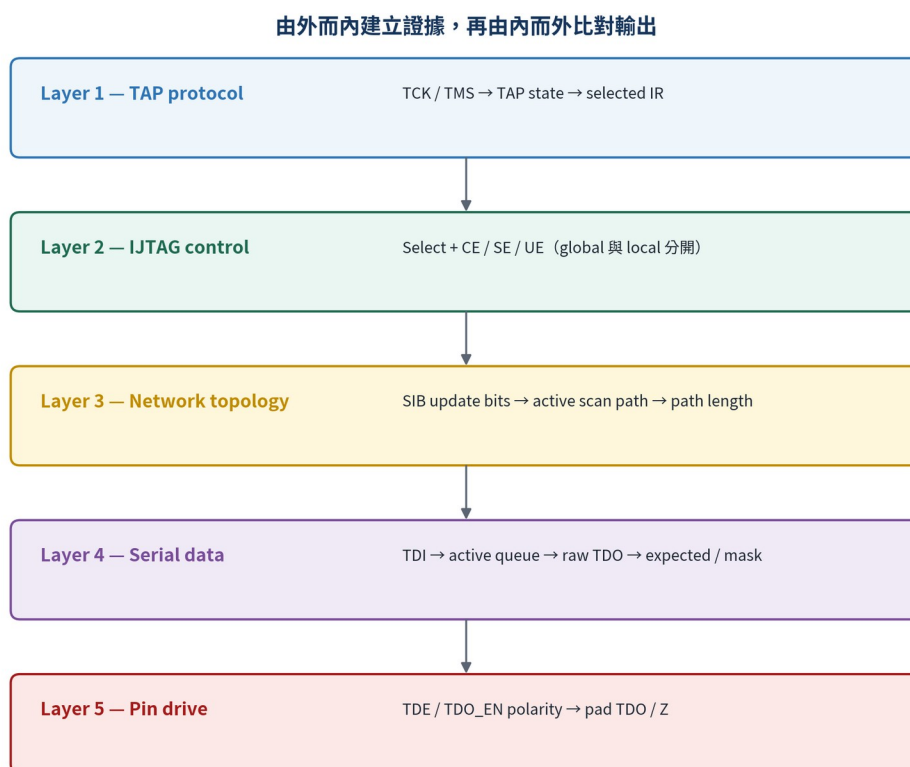


圖 1 五層證據鏈：先決定 TAP 狀態，再追 IJTAG control、SIB topology、serial data 與實際 pad drive。

一句話判定原則 TAP sequence 合法、當下 active path 長度正確、逐 bit 的 reference model 與有效 TDO compare 完全相符，三者同時成立，才足以說自動產生的 pattern 是對的。

你目前幾個疑問的直接答案

疑問	直接答案	真正應檢查的點
為什麼一開始有 CE、UE 各一個 cycle？	正常。Capture-DR 與 Update-DR 是 TAP 的暫態 state，標準 DR transaction 必須經過。	看有效 edge 上是否 capture/update，不要只看矩形是否與 state label 完美對齊。
為什麼 SIB 一直跳？	如果看到的是 SIB shift stage，SE 期間跳動正常；真正控制路徑的是 update/select stage。	SIB update/select 應只在 UpdateEn event 後改變。
為什麼有些訊號在 negedge 動？	常見於 TDO retiming、TDO output enable 或 update latch，用來避免與 rising-edge shift 競爭。	先辨識該 flop 是 shift、update 還是 output retime；1687 不要求特定 RTL always block 寫法。
TDO_EN 為什麼在 SE 結束後才拉高？	高度可能是低有效 output-disable/OE_N，或經 falling-edge retime；拉高可能代表關閉輸出。	追到 pad primitive，查 enable 極性與 pad_tdo 是否為 Z。
CE/SE/UE 一次只能做一個動作嗎？	正常 TAP access 中，對同一個 scan register 的 event 是互斥的；但 SE 每拍同時移入 TDI、移出 TDO。	在執行 edge 判斷，而非只看 enable pulse 是否視覺重疊。

文件結構

1. 介紹：IEEE 1687 解決什麼問題、與 IEEE 1149.1 的關係、每個訊號屬於哪一層。
2. 運作流程：reset、選 instruction、Capture-Shift-Update、開關 SIB、存取 TDR、nested SIB。
3. 波形解析：edge timing、SIB 跳動、TDO/TDO_EN、local/global enable、常見 viewer 錯覺。
4. Trace SOP：如何從 ICL 與 waveform 重建 active path，逐 bit 比對 TDI/TDO。
5. 完整範例：1-bit SIB + 4-bit TDR 的手算表，以及 nested SIB 的 shift-length 推導。
6. 驗證與除錯：negative control、over-shifting、失敗症狀對應原因、sign-off checklist。

1. IEEE 1687 與波形觀察模型

1.1 IEEE 1687 的定位

IEEE 1687-2014 (IJTAG) 定義的是晶片內 embedded instrument 的存取與操作方法，包含可重組的 scan access architecture、Instrument Connectivity Language (ICL) 與 Procedural Description Language (PDL)。它的重點是讓 instrument 與 access network 可描述、可整合、可 retarget，而不是規定 instrument 本身必須做什麼功能。[1][2]

1687 network 可以由 IEEE 1149.1 TAP hosting，但 1687 的概念起點是 network gateway/interface，不等於外部一定只能使用某一種 pad-level JTAG 實作。對目前使用 Tessent TAP 的情境，可把 1149.1 視為外部 transport，把 1687 視為晶片內部可重組的 instrument network。[1][7]

標準與實作要分開 IEEE 1687 規定介面語意與可觀察行為，但不要求工具必須用某一種 SIB gate-level 結構、某個訊號名稱，或固定用 posedge/negedge 寫 RTL。只要 IO protocol 與描述模型一致，實作可以不同。[4]

1.2 波形必須分成五層

層級	主要訊號／物件	該層要回答的問題
TAP protocol	TCK, TMS, TRST_N, tap_state, current_ir	目前是 IR scan 還是 DR scan？哪個 instruction 選中 IJTAG access register？
IJTAG control	Select, CE, SE, UE, Reset；global/local 版本	哪個 interface 在這個 edge 真正收到 capture、shift 或 update event？
Network topology	SIB shift、SIB update/select、ScanMux	當下 active scan path 包含哪些 scan elements？總長度是多少？
Serial data	TDI, internal SI/SO, raw TDO, expected, mask	每個 shift edge 前的 tail bit 是什麼？新 bit 如何進入 queue？
Pin drive	TDE, TDO_EN/OE_N, tdo_data,	內部資料是否真的被輸出到 pad？enable 極性為何？

層級	主要訊號／物件	該層要回答的問題
	pad_tdo	

1.3 1687 separable interface 的核心訊號

常見 1687 interface 可抽象成八個訊號：TCK、Reset、Select、CaptureEn、ShiftEn、UpdateEn、ScanIn 與 ScanOut。這些是 network/instrument interface 的語意；外部 JTAG 的 TMS、TDI、TDO 則屬於 hosting TAP。研究論文與 IJTAG tutorial 都以這組 separable interface 說明 IP 的可重用性。[2][7][9]

訊號	方向	標準語意	波形判讀重點
TCK	Input	Capture/Shift/Update 的時間參考。	先標出所有 rising/falling edges，再談其他訊號。
Reset	Input	把 network control/update state 帶到已知 reset value。	SIB update state 通常需有已知 reset 值；不要只看 shift stage。
Select	Input	選中此 scan interface／segment。	local CE/SE/UE 是否有效，通常還要搭配 Select。
CaptureEn / CE	Input	把 capture source 載入 shift stage。	通常只在 Capture event 作用一次；不代表正在 serial output。
ShiftEn / SE	Input	使 active scan elements 沿 SI→SO shift。	同一個 edge 同時 TDI 移入、tail bit 移出。
UpdateEn / UE	Input	把 shift stage 的內容套用到 update stage／instrument。	更新後才改變 instrument control 或 SIB select。
ScanIn	Input	此 interface 的 serial input。	不一定直接等於 chip TDI，可能經上游 SIB/ScanMux。
ScanOut	Output	此 interface 的 serial output。	不等於 pad TDO；中間可能還有 mux、retime 與 output enable。

1.4 哪些名稱不是 IEEE 1687 的通用 port

``TDE``、``TDO\_EN``、``ijtag\_ce``、``ijtag\_se``、``ijtag\_ue`` 可能是 Tessent 產生 RTL 的 internal net 或 top-level convention。尤其 ``TDE`` 不是 IEEE 1687 separable interface 的標準

port name；它可能代表 TDO data enable、pre-retimed enable、instruction/data select 的組合 decode，或 pad wrapper 的控制。必須從 netlist/RTL fan-in 與 fan-out 判斷。

```
rg -n '\b(tde|tdo_en|tdo_oe|oe_n|ijtag_ce|ijtag_se|ijtag_ue)\b' generated_rtl/
rg -n '\b(always_ff|always)\s*@\s*\s*(negedge\s+tck)' generated_rtl/
rg -n '\b(PAD|OEN|OE|OE_N|TRI|TDO)\b' generated_rtl/
```

- 名稱帶 `\_en` 不保證 active-high；pad cell 常以 OEN/OE_N 表示低有效 output enable。
- 內部 `tdo\_data` 有 0/1 不代表外部 pin 正在 drive；只有 pad model 與 enable 合併後才會出現有效 0/1 或 Z。
- 若同時存在 `tde` 與 `tdo\_en`，常見關係是前者為 combinational decode、後者為 falling-edge retimed 或極性轉換後的 pad control，但這只是待驗證假設。

2. 1687 基本硬體行為

2.1 TDR：shift stage 與 update stage 必須分開

一個可讀寫的 TDR 通常包含 capture/shift stage 與穩定的 update stage。Capture 將 instrument status 載入 shift stage；Shift 只移動 shift stage；Update 才將新值套用到 instrument control。將 shift 與 update 分開，可以確保長串資料移動期間不會讓 instrument control 跟著每一個 TDI bit 抖動。[3]

```
// Conceptual model only – not exact Tessent RTL
on Capture event: shift_reg <- capture_source
on Shift event:   {shift_reg, ScanOut} <- {ScanIn, shift_reg}
on Update event:  update_reg <- shift_reg
instrument_control <- update_reg
```

TDR 類型	Capture	Shift	Update	常見用途
Shift-only	無	有	無	pipeline / timing normalization
Shift-Capture	有	有	無	read-only status
Shift-Update	無或 self-capture	有	有	write-only control
Shift-Capture-Update	有	有	有	read/write register

2.2 SIB：為什麼一定要看兩個 state

SIB 是用來動態插入或移除 network segment 的 scan element。典型 SIB 由一個可 shift 的 bit、一個穩定的 update/select bit，以及一個 scan mux 組成。公開的 IEEE 1687 working-group material 明確區分 shift bits 與 update bits：shift bits 可在 scan 過程中改變，update bits 為 instrument/network 提供穩定控制，且只在 UpdateEn event 改變。[3]

```
SIB_shift : capture / shift 用的暫存值
SIB_update : 穩定的 path-control 值

if SIB_update == 0:
    ScanOut <- SIB_shift           // child segment bypassed
else:
    ScanOut <- child_segment.ScanOut // child segment inserted
    child_segment.ScanIn <- SIB_shift
```

核心辨識 波形中叫 `sib` 的 net 若接到 shift flop Q，SE 期間跳動正常；若它接到 ScanMux select/child Select，則應由 update stage 驅動，不能在每個 shift bit 跟著 TDI 跳。

- 把 1 shift 到 SIB shift stage，不會立即打開 segment；必須等 Update event。
- SIB 關閉時，child TDR 通常不在 active path，也不應收到有效 local shift event。
- 關閉 parent SIB 不必然清除 child SIB 的 update state；重新打開 parent 後，child path 可能依保留狀態立即恢復。是否 reset/retain 依 ICL ResetValue 與 RTL 而定。
- 可以在 path 仍開啟的那一次 Shift 中同時寫 TDR 與把 SIB shift bit 寫成 0；到 Update event 時 TDR 新值生效，SIB 同時關閉。

2.3 Global control 與 local control

TAP 可能產生 global CE/SE/UE，但 instrument 是否真正動作還取決於該 scan interface 是否被 Select。概念上常見 gating 如下，但實際 RTL 可能把 gating 放在 clock、data mux 或 enable tree 中。

```
local_ce = global_ce & local_select
local_se = global_se & local_select
local_ue = global_ue & local_select
```

```
// 等價實作也可能是：global event 仍存在，
// 但 local clock gate 或 data mux 讓未選 segment 保持。
```

因此，當 SIB 關閉時看到 global `ijtag\_se` 正常跳動，不代表 SIB 後面的 TDR 也在 shift。真正要找的是 local interface enable、local clock 或 TDR shift flop 的實際 data movement。

3. 一次完整 IJTAG 存取的運作流程

3.1 高階流程

1. Reset network：確認 TAP state、SIB update reset values 與 instrument safe state。
2. IR scan：把可存取 IJTAG network 的 instruction 載入 TAP instruction register。
3. DR scan A：先配置 SIB／ScanMux，建立下一次 access 所需的 active path。
4. Update-DR：SIB update/select state 改變，新的 path 才生效。
5. DR scan B：對已打開的 TDR 做 capture、shift 與 update。
6. 需要時關閉 segment，或保留開啟以進行下一次存取。
7. 由 expected TDO 與 mask 驗證讀回資料；由 update stage／instrument port 驗證寫入結果。

3.2 DR transaction : $CE \times 1 \rightarrow SE \times N \rightarrow UE \times 1$

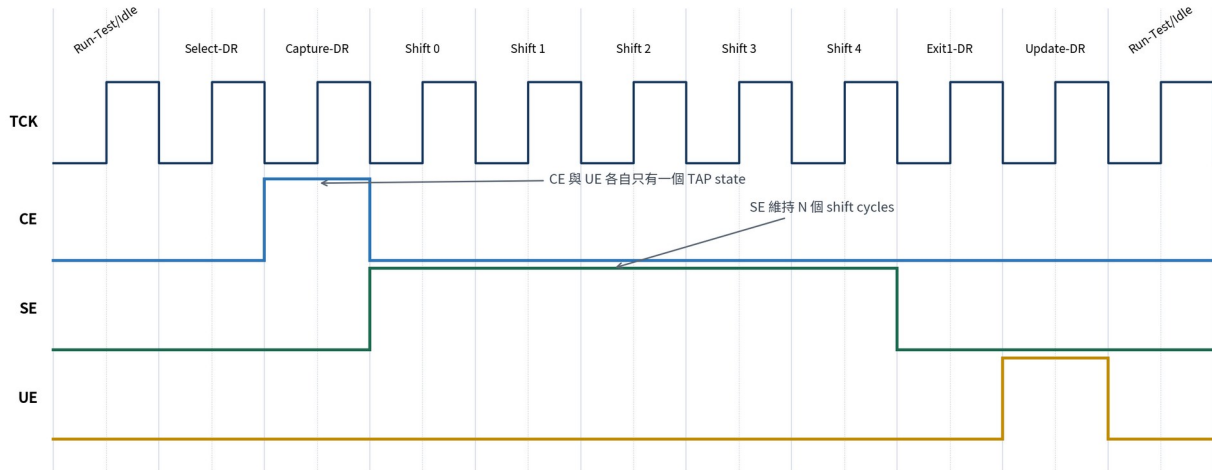


圖 2 典型 1149.1-hosted DR transaction。CE 與 UE 各對應一個暫態 state；SE 持續 active path 所需的 N 個 shift cycles。

對正常、不中途 Pause 的 DR scan，可用 `Capture-DR → Shift-DR × N → Exit1-DR → Update-DR` 理解。Capture-DR 與 Update-DR 不能停留，所以各自只佔一個 TAP state interval；Shift-DR 可以透過 TMS=0 自迴圈 N 次。[5]

為什麼 write-only 也會經過 CE？ TAP protocol 仍然經過 Capture-DR。某個 write-only TDR 的 local CaptureEn 可以被關閉、忽略或 self-capture，但不能因此假設整個 TAP 不會出現 Capture-DR/global CE。

3.3 CE、SE、UE 是否只能做一個動作

在標準 1149.1-hosted transaction 中，對同一個被選 scan register 而言，Capture、Shift、Update 是不同 event；正常執行 edge 上只應發生其中一種 state transition。這不等於整個晶片只能有一個 register 動作，也不等於 Shift 只能輸入或只能輸出。

Event	Shift stage	Update/instrument stage	Serial pins
CE	由 capture source 平行載入	維持既有輸出	TDO 可能有內部組合值，但不視為有效 shift compare

Event	Shift stage	Update/instrument stage	Serial pins
SE	整條 active path 同時 shift	維持既有輸出	TDI 移入與 TDO 移出同時發生
UE	通常 hold	由 shift stage 更新	不做 serial shift
皆為 0	hold	hold	TDO 是否 drive 由 TAP/pad 規則決定

假設 active path 是 `SIB → TDR\_A → TDR\_B`，一次 SE edge 會讓所有 active elements 同時向 TDO 方向移動。TDI bit 進入 chain 頭端，而 shift 前最靠近 TDO 的 tail bit 成為該拍讀出值。這是一個 full-duplex serial shift，而不是 read/write 二選一。

3.4 打開一個 SIB 為什麼至少需要兩次 DR scan

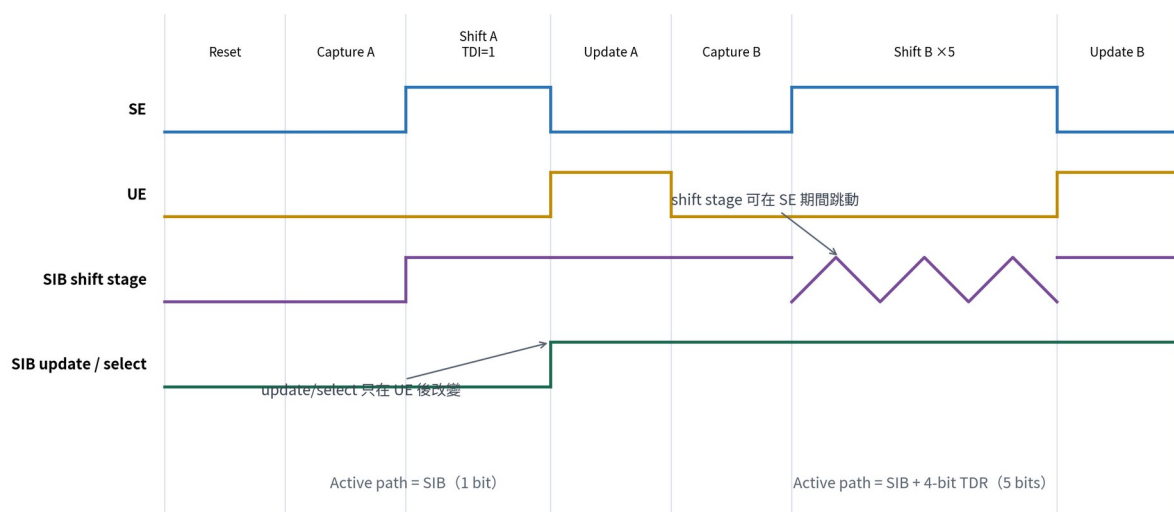


圖 3 第一次 DR scan 只把 SIB 的 update/select 設為 1；下一次 Capture/Shift 才會看到加長後的 SIB + TDR path。

階段	SIB shift	SIB update/select	Active path	可否存取 TDR
Reset	0	0	SIB (1 bit)	否
Capture A	capture reset/update value	0	SIB (1 bit)	否
Shift A : TDI=1	變成 1	0	仍是 SIB (1 bit)	否
Update A	1	變成 1	此刻 path control 改變	從下一次 scan 起可存取
Capture/Shift	可能再跳動	保持 1	SIB + TDR	是

階段	SIB shift	SIB update/select	Active path	可否存取 TDR
B				

3.5 Nested SIB 的開啟順序

假設結構為 `Parent SIB → Child SIB → 8-bit TDR`，reset 後兩個 SIB 都關閉，最保守的存取序列如下。

Transaction	Scan 前 update state	當下 path	Shift length	Update 後
A：開 Parent	P=0	P	1	P=1
B：開 Child	P=1, C=0	P + C	2	P=1, C=1
C：存取 TDR	P=1, C=1	P + C + TDR[7:0]	10	依新資料／是否關 SIB

重點是每一次 scan 的長度由 scan 開始前的 update/select state 決定，不是由本次 shift 中正在寫入的 SIB shift bit 決定。若把 Child=1 shift 進去後立刻假設同一筆 scan 後半段已包含 8-bit TDR，就會整串 bit 對齊錯誤。

4. Clock edge 與 waveform viewer 的判讀

4.1 rising edge 與 falling edge 的角色

對 1149.1 TAP，TMS 與 TDI 在 TCK rising edge 取樣；TDO 通常在 falling edge 改變，以提供約半個 TCK 的穩定時間給下一個 rising edge 的接收端。[5][6] 這是理解「SE 已經降了，但 TDO_EN 又晚半拍變化」的核心。

Edge	常見動作	Trace 時的注意事項
TCK rising	TMS/TDI 取樣；TAP state transition；capture 或 shift stage 動作	對每個 rising edge 記錄 state_before、TMS、state_after、TDI 與 tail bit。
TCK falling	TDO data retiming／改變；TDO output	確認是標準要求、TAP implementation 還是

Edge	常見動作	Trace 時的注意事項
	enable 切換；某些 update latch 更新	tool/pad-specific retiming。

4.2 為什麼 state label 常看起來差一拍

不同 waveform viewer 或 RTL probe 可能顯示「rising edge 後的 state register 值」，但 register action 是由 edge 前的 current state decode 觸發。許多 1149.1 device 說明會描述：Capture 動作發生在離開 Capture-DR 的 rising edge；第一個 shift 發生在進入 Shift-DR 後的下一個 rising edge；最後一個 shift 發生在離開 Shift-DR 的 rising edge。[10]

不要只靠色塊對齊 建立 edge table：對每個 rising edge 同時記錄 `state\_before`、`TMS\_sampled`、`state\_after`、`CE/SE/UE sampled` 與 register 變化。只要 action edge 與協定一致，enable pulse 在 viewer 上視覺上跨到相鄰 state 不一定是錯。

```
edge k:
  state_before = Shift-DR
  TMS_sampled  = 1
  action       = last shift (TDI sampled, old tail read)
  state_after  = Exit1-DR

falling edge k+0.5:
  TDO driver may be disabled / tri-stated
```

4.3 哪些 negedge 行為是合理的

- TDO retiming register：把 selected scan-out 在 falling edge 送到 TDO data path。
- TDO output enable retiming：在進入 Shift-IR/Shift-DR 後的第一個 falling edge enable，在離開 shift state 後的第一個 falling edge disable。
- Update latch：有些 TAP/TDR implementation 在 Update-DR 的 falling edge 套用新值。
- 反相或 gated local clock：RTL 可能看似在 raw TCK negedge，但實際代表另一個 local event。

真正值得警覺的是：一個被定義為 shift stage 的 TDR/SIB flop，在沒有任何說明的情況下直接於 raw TCK negedge shift，造成與 TDI 驅動／TDO compare timing 不一致。遇到這種情況不能只看 always block，還要往上追 local clock 的來源與 testbench 在何時 drive TDI。

5. TDO、TDE 與 TDO_EN

5.1 內部資料與外部輸出是兩條問題

```
TDR/SIB scan_out
-> IJTAG network ScanOut
-> TAP selected_data_out / raw_tdo
-> optional negedge retiming
-> tdo_data -----> pad data input

Shift-IR/Shift-DR decode
-> TDE / pre-enable
-> optional negedge retiming / polarity convert
-> TDO_EN or OE_N -----> pad output-enable input
```

即使 pad output disabled，內部 `tdo\_data` 仍可正常顯示 TDR 的 0/1。若 testbench 沒有 instantiate tri-state pad model，top-level 的 `tdo` net 也可能永遠顯示資料而不會顯示 Z。因此，要證明外部真的輸出，必須看 pad output 或用 pad primitive truth table 推導。

5.2 你看到「SE 結束後 TDO_EN 才拉高」的最可能解釋

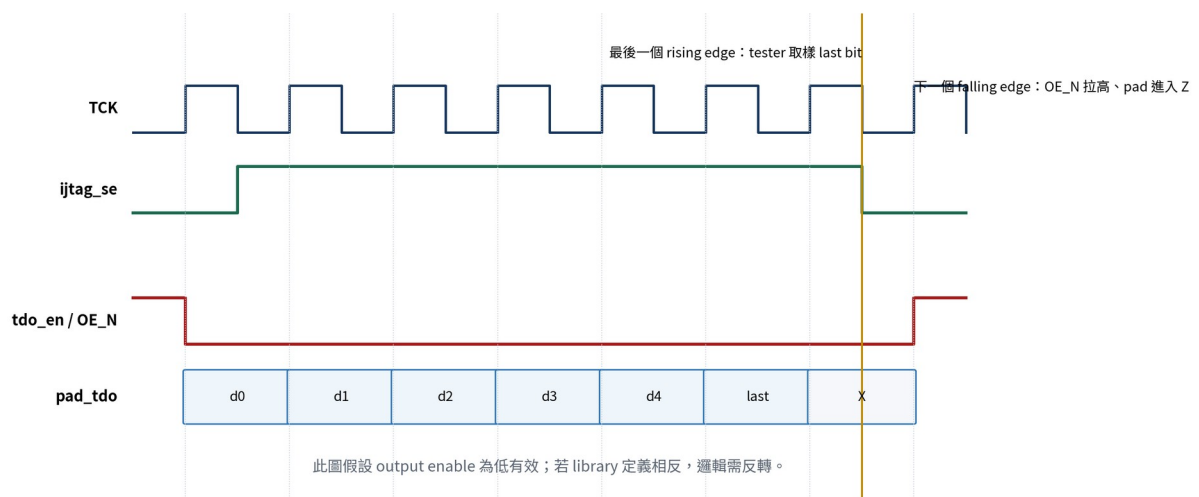


圖4 若 `tdo\_en` 實際接到低有效 OE_N，Shift 期間為 0 才是 enable；SE 結束後在 falling edge 拉高代表關閉 driver。

若 `ijtag\_se=1` 期間 TDO 已依序出現 TDR bits，而 `tdo\_en` 在 SE 結束後由 0 拉到 1，最合理的第一假設不是「enable 太晚」，而是 `tdo\_en` 實際為 output-disable/active-low OEN。最後一個 shift 的 rising edge 已經完成 tester 取樣，下一個 falling edge 才關閉 driver，正好符合 JTAG 常見 pin timing。[6]

要查的證據	看到什麼代表 active-low	看到什麼代表 active-high
Pad assign	`pad_tdo = tdo_en ? 1'bz : tdo_data`	`pad_tdo = tdo_en ? tdo_data : 1'bz`
Pad primitive pin	接到 OEN / OE_N，library 定義 0=drive	接到 OE，library 定義 1=drive
Shift 期間 pad_tdo	tdo_en=0 且 pad_tdo 為 0/1	tdo_en=1 且 pad_tdo 為 0/1
離開 Shift 後	tdo_en=1 且 pad_tdo=Z	tdo_en=0 且 pad_tdo=Z

5.3 TDO 有值不等於該拍要 compare

Pattern 中的 TDO 是否有效，至少要同時考慮：目前是否為有效 shift state、外部 driver 是否 enable、pattern compare mask 是否為 care。Capture、Exit、Update 或 output disabled 期間，內部 TDO data path 出現 0/1 通常沒有比較意義。

```
valid_compare = is_shift_state
                & pad_driver_enabled
                & pattern_tdo_mask_is_care

if valid_compare:
    assert actual_pad_tdo == expected_tdo
```

TDE 的處理原則 在沒有 generated RTL 或 library 定義前，不要把 `TDE` 翻成 IEEE 1687 的標準功能。先追它是否為 `(shift\_ir | shift\_dr)` 的 decode、是否進入 negedge flop、是否反相後成為 pad enable。

6. SIB 為什麼會跳來跳去

6.1 三種可能看到的 `sib`

Waveform net	SE 期間預期	UE 時預期	判定
sib_shift / scan_cell_q	隨 TDI 與上游資料移動，可多次 0↔1	通常保持最後 shift 值	跳動正常
sib_update / hold_q	保持不變	只在有效 update event 改變	SE 期間跳動才可疑
to_select / segment_select	通常等於 sib_update，保持	update 後影響 child path	若直接跟 sib_shift 抖動，多半接錯或看錯 net
scan_mux_output	會因 serial data 自然變化	依 select 決定下一次 path	不能當成 SIB state

6.2 正常 open sequence

1. Reset：`sib\_update=0`，child segment bypass。
2. Capture：`sib\_shift` 載入 reset/update/capture source；`sib\_update` 仍為 0。
3. Shift：把 TDI=1 移入 `sib\_shift`；此時 active path 仍未改變。
4. Update：`sib\_update <- sib\_shift = 1`；child select 變為 1。
5. 下一次 Capture/Shift：active path 才包含 child TDR。

6.3 哪些 SIB 現象真的可能是錯

- `segment\_select` 在 SE 高期間每拍改變，造成 active path 在同一筆 scan 中動態伸縮。
- UE=0 時 update flop 改變，或 UE 發生後 update flop 沒有取得最後的 shift value。
- SIB reset 後不是 ICL 宣告的 ResetValue，導致工具預期 path 與 RTL path 不一致。
- ICL 的 ScanMux SelectedBy 指向某個 update register，但 RTL mux 由 shift flop 驅動。

- parent Select 已關閉，child local shift flop 仍在移動，且不是刻意設計的 broadcast/always-on path。

7. 實際 Trace SOP：從 waveform 證明 pattern

7.1 建議 waveform 分組

Group	至少加入的訊號	目的
A — TAP	tck, tms, trst_n, tap_state, ir_shift, current_ir	重建 state sequence 與 selected instruction
B — IJTAG global	ijtag_select, global_ce, global_se, global_ue, reset	證明 TAP decode 正常
C — SIB/local	每個 sib_shift, sib_update, to_select, local_ce/se/ue	重建可變 path 與 gating
D — Serial path	tdi, 各段 scan_in/out, raw_tdo, retimed_tdo	逐段定位資料開始錯的位置
E — Pin/compare	tde, tdo_en, pad_tdo, expected_tdo, mask	只在真正有效的 compare window 判斷
F — Instrument	capture_source, update_q, functional outputs/status	驗證 CE 讀回與 UE 寫入的平行端行為

7.2 Step 1：從 TMS 重建 TAP state

不要先相信 waveform 中名為 `tap\_state` 的字串，先用 TMS 在每個 rising edge 的取樣值自行重建一次。確認 reset 後 state 已知，並檢查 IR/DR path 是否符合 1149.1 state machine。

記錄欄位：

edge_id | time | state_before | TMS | state_after | TDI | TDO | note

DR scan 的核心序列：

Select-DR -> Capture-DR -> Shift-DR x N -> Exit1-DR -> Update-DR

- 若出現 Pause-DR，SE 應停止；resume 後 path state 必須接續，不可重新 capture。
- 若同一 waveform 同時有 IR scan 與 DR scan，先切段，避免把 Shift-IR 的 TDO bits 當成 TDR response。
- 確認 current IR 的 update 時點；IR shift register 的中間值不應立即改變 DR selection。

7.3 Step 2：確認 selected instruction 與 IJTAG Select

追 `current\_ir` / instruction update stage 到 decoder，再到 IJTAG network Select。只有 instruction update 後，後續 DR scan 才應連到 IJTAG network。若 IR shift 期間 IJTAG Select 跟著 instruction shift bits 抖動，表示你看到的是 shift stage decode 或 RTL 有問題。

常見誤判 看見 `ir\_shift` 途中某些 bit pattern 恰好等於 IJTAG opcode，就以為 IJTAG network 提前被選。真正 decoder 應由 IR update stage 驅動，而非正在 shift 的暫存內容。

7.4 Step 3：把 global CE/SE/UE 映射到 TAP event

TAP event	預期 global control	主要 register action
Capture-DR	CE=1, SE=0, UE=0	選中 DR 的 capture stage 載入
Shift-DR	CE=0, SE=1, UE=0	active path 每個 shift edge 移動一 bit
Update-DR	CE=0, SE=0, UE=1	update stage / SIB select 套用新值
其他 DR states	通常三者皆 0	hold ; TDO enable 依 TAP state/pad implementation

若 enable pulse 因 combinational decode 在 edge 前後有短暫重疊，先看 register 實際在哪個 edge 動作。對同步功能而言，sampled value 與 action edge 比類比視覺上的 pulse overlap 更重要。若重疊跨越有效 edge 並導致同一 register 同時 capture/shift，才是實質風險。

7.5 Step 4：逐顆維護 SIB shift/update state

建立一張 SIB state table。每個 Capture event 更新 shift state；每個 Shift event 只移動 shift state；每個 Update event 才更新 update/select state。路徑重建一律使用 update/select state。

Edge	Event	SIB shift before	TDI	SIB shift after	SIB update after	Path for next scan
0	Reset	-	-	0	0	SIB only
1	Capture	0	-	0	0	SIB only

Edge	Event	SIB shift before	TDI	SIB shift after	SIB update after	Path for next scan
2	Shift	0	1	1	0	SIB only
3	Update	1	-	1	1	SIB + child

7.6 Step 5：依 ICL connectivity 重建 active path

Active path 不能只靠 RTL instance name 排序。應從 ICL 的 ScanInSource、ScanOutPort Source、ScanMux/SelectedBy 與各 SIB update state 推導。路徑是一個由 scan primary input 開始、沿選中的 mux branch 經過 scan elements、最後到 scan primary output 的有序序列。

[4]

```
active_path = walk_from(primary_scan_in)
while current_node != primary_scan_out:
    if node is ScanMux:
        branch = value_of(node.SelectedBy.update_state)
        append every ScanRegister bit in serial order
        follow ScanOutPort.Source
```

```
N = sum(bit_width(element) for element in active_path)
```

- 每次 DR scan 開始前固定一份 path snapshot；正常 SIB 不會在同一筆 Shift 中立即改變 mux branch。
- 紀錄 path 中每個 register 的 serial bit order，不能只記總長度。
- 若 ICL 與 RTL path 不一致，pattern 可能在固定延遲處整串 miscompare，或在打開 SIB 後才開始錯。

7.7 Step 6：建立 serial queue reference model

最不容易搞錯 bit order 的方法，是不先使用 MSB-first/LSB-first 口語，而是定義一個 queue：

「q[0]」最靠近 TDI，「q[-1]」最靠近 TDO。每一個 Shift event 前，預期 TDO 就是舊的「q[-1]」；Shift 後把新 TDI 插到「q[0]」。

```

Capture event:
    q = capture_bits_of(active_path)          # q[0] near TDI; q[-1] near TDO

Shift event:
    expected_tdo = q[-1]                      # value before the shift
    q = [sampled_tdi] + q[:-1]

Update event:
    apply q slices to update stages
    recompute next active_path from SIB update values

```

TDO 的採樣相位 若 DUT 在 falling edge 更新 TDO，tester 通常在下一個 rising edge 前後的 compare window 取樣。Reference model 的 `expected\_tdo=q[-1]` 必須對到實際 tester compare edge；不要把 shift 後的新 tail 與同一拍 compare 混在一起。

7.8 Step 7：最後才套用 TDO enable 與 compare mask

1. 確認 `tdo\_en` 極性與 pad primitive。
2. 確認 enable 是否在第一個 Shift falling edge 才生效，以及最後一個 Shift rising edge 後才關閉。
3. 只在 pad driver enabled 且 pattern mask=care 的 bit 比對 actual/expected。
4. 將 first mismatch 的 bit index 映射回 active path element；再檢查該 element 的 SI、Q、SO。

7.9 建議逐拍 trace 表

欄位	內容
Protocol	edge/time、state_before、TMS、state_after、selected IR
Control	global Select/CE/SE/UE、local Select/CE/SE/UE
Topology	所有 SIB shift/update、active path、expected length
Data	TDI、queue before、expected TDO、queue after
Pin	raw TDO、retimed TDO、TDE、TDO_EN polarity、pad TDO
Compare	expected value、mask、actual value、pass/fail

8. Worked Example : 1-bit SIB + 4-bit TDR

8.1 假設與 bit ordering

假設 path 關閉時只有 1-bit SIB；打開後為 `SIB, TDR[3], TDR[2], TDR[1], TDR[0]`，其中左側最靠近 TDI，`TDR[0]` 最靠近 TDO。定義 queue 為 `[near TDI ... near TDO]`。

- Reset：SIB update=0。
- Transaction A：送入 TDI=1，Update 後打開 SIB。
- Transaction B 的 Capture：SIB capture=1，TDR capture value=4'b1010。
- Transaction B 希望 Update 後：SIB 保持 1，TDR 寫入 4'b0110。

8.2 Transaction A：開 SIB

Event	q before	TDI	Expected TDO	q after	SIB update
Capture	[0]	-	-	[0]	0
Shift	[0]	1	0	[1]	0
Update	[1]	-	-	[1]	1

注意：Shift 完成時 `sib\_shift=1`，但直到 Update event，`sib\_update` 才變為 1；

Transaction A 的 shift length 從頭到尾仍是 1。

8.3 Transaction B：讀出 1010，同時寫入 0110

Capture 後 queue 為 `[SIB, b3, b2, b1, b0] = [1,1,0,1,0]`。要讓 Shift 結束後 queue 變成 `[1,0,1,1,0]`，TDI 必須依時間先後送 `[0,1,1,0,1]`，也就是從靠近 TDO 的目標 bit 開始送，最後才送靠近 TDI 的 SIB bit。

Shift #	q before	TDI	Expected TDO	q after
0	[1,1,0,1,0]	0	0 = old b0	[0,1,1,0,1]

Shift #	q before	TDI	Expected TDO	q after
1	[0,1,1,0,1]	1	1 = old b1	[1,0,1,1,0]
2	[1,0,1,1,0]	1	0 = old b2	[1,1,0,1,1]
3	[1,1,0,1,1]	0	1 = old b3	[0,1,1,0,1]
4	[0,1,1,0,1]	1	1 = old SIB	[1,0,1,1,0]

因此 TDO 的時間序列為 `[0,1,0,1,1]`：先讀出 TDR[0..3]，最後讀出 SIB。Update 後，SIB 仍為 1，而 TDR update value 為 4'b0110。

與工具 vector 的 bit order 某些 STIL/WGL/Tessent 顯示會把字串以 MSB→LSB 寫出，但實際 pin drive 仍有時間先後。報告時建議同時標註「vector display order」與「TDI chronological order」，避免用一句 LSB-first 造成歧義。

8.4 若本次同時關閉 SIB

只要把最後一個 TDI（對應 SIB）由 1 改成 0，Shift 期間 path 仍然是 5 bits，因為 mux 由舊的 `sib\_update=1` 控制。Update event 後 SIB 才關閉，下一次 scan length 變回 1。若 waveform 在第 5 個 shift edge 前就突然縮短，才表示 mux 可能誤用 shift stage。

9. 如何證明自動產生的 pattern 是正確的

9.1 四層 sign-off evidence

證據	Pass criterion	產出
Protocol proof	從 TMS 重建的 TAP state 合法；IR/DR scan 分段正確。	edge-by-edge TAP table
Topology proof	每次 scan 的 active path 由 scan 前 SIB update state 唯一決定；shift count=N。	path list + length
Data proof	Capture source、TDI queue model 與 expected TDO 逐 bit 一致。	serial trace / first mismatch
Pin proof	只在 driver enabled、compare mask=care 的 window 比	pad timing + polarity evidence

證據	Pass criterion	產出
	對 pad TDO。	

如果只有 simulation pass，但沒有獨立重建 path 與 serial queue，最多能說「Tessent 產生的 pattern 與同一套模型相容」，還不足以排除 ICL 與 RTL 同時錯、或 testbench compare window 錯位。

9.2 建議的 negative controls

1. 反轉開 SIB 的那一個 TDI bit：下一次 path length 應維持短鏈，原本 TDR response 應 miscompare。
2. 刻意少 shift 1 bit：Update 後 SIB/TDR 應產生可預測的 bit displacement，而不是仍然 pass。
3. 刻意多 shift 1 bit：marker pattern 到達 TDO 的時間應晚/早一拍，證明 checker 對 path length 敏感。
4. 對 expected TDO 的一個 care bit 反轉：compare 必須 fail；若仍 pass，檢查 mask 或 compare phase。
5. 在 pad model 中確認 output disabled 時 pad_tdo=Z；若內部 tdo_data 仍有值但 pad_tdo=Z，證明兩層觀察點分離正確。

Mentor/Siemens 公開的 IJTAG network diagnosis tutorial 建議使用 over-shifting：送入比預期 path 更長的 marker pattern，觀察它在 TDO 出現得太早或太晚，以診斷 SIB stuck-deassert / stuck-assert，以及 ICL 與實際 TDR 長度不一致。[4]

9.3 Over-shifting 判讀

現象	可能原因	marker 到達時間
SIB stuck closed / deassert	預期 child segment 存在，但實際被 bypass	比預期早
SIB stuck open / assert	預期短鏈，但實際 child segment 仍插入	比預期晚

現象	可能原因	marker 到達時間
TDR 實際長度小於 ICL	RTL 少 bit / ICL 多宣告	比預期早
TDR 實際長度大於 ICL	RTL 多 bit / ICL 少宣告	比預期晚
一路固定 0 或 1	stuck-at、錯誤 mux branch、reset/enable 問題	marker 可能不出現

10. 常見失敗症狀與定位

症狀	先檢查	常見根因
第一個 TDO bit 就錯	Capture source、TDO compare phase、第一個 falling-edge enable	把 shift 後 tail 當成 shift 前 tail；TDO retime 錯半拍
打開 SIB 前都對，打開後整串錯	SIB update/select、path order、TDR width	ICL/RTL connectivity 不一致；把 shift bit 當 select
固定晚/早 k bits	active path 總長度與各 segment width	漏算 SIB、bypass bit、pipeline bit，或 TDR width 不符
只有最後一個 bit 錯	離開 Shift-DR 的最後 rising edge 與 TDO disable falling edge	checker 太早關 compare；TDO_EN phase 錯
TDO 有資料但 pad 為 Z	TDO_EN 極性與 pad truth table	只看 internal tdo_data；enable 未打開
TDO_EN 看似 SE 結束後才高	它是否為 OE_N、是否 negedge retime	把 active-low disable 誤當 active-high enable
SIB 在 shift 中一直跳	net 是 shift_q 還是 update_q	正常 shift data，或 mux 誤接 shift_q
local CE/SE/UE 全沒動	parent Select、IR decoder、SIB update state	segment 未被選、instruction 未 update
global CE/SE/UE 正常但 TDR 不動	local gating / clock gate	SIB 關閉或 local Select=0，可能是正常
UE 後 instrument 值沒變	TDR 是否有 update stage、local UE、bit slice mapping	write-only/update port 未接；UE edge 對錯
Capture 後讀回全 X	instrument capture source、reset、clock/domain crossing	functional source 未初始化；capture mask 應設 X

10.1 遇到 first mismatch 的縮小範圍法

1. 確認 mismatch bit 是否在 care window；若 mask=don't-care，先修 checker/report。

2. 把 mismatch 的 shift index 映射到 queue tail 與 active path element。
3. 同一拍比較該 element 的 Q、SO、下游 SI 與 raw TDO，找第一個不連續點。
4. 若 element Q 正確但 SO 錯，查 scan mux/select；若 Q 已錯，往前查 capture/shift input。
5. 若 raw TDO 正確但 pad_tdo 錯，問題在 retime、enable 或 pad；不要回頭改 SIB。

11. 報告時可以怎麼講

11.1 三分鐘版說法

「我不是直接看 Tessent pattern pass 就認定正確，而是分三層驗證。第一層由 TMS 重建 TAP state，確認 Capture、Shift、Update sequence；第二層以每個 SIB 的 update state 重建 active scan path，確認 shift cycles 等於 path length；第三層依 ICL serial connectivity 建立 queue model，逐 bit 預測 TDO，並只在 TDO driver enable 與 compare mask 有效時比對。SIB shift bit 在 Shift 期間跳動是正常的，真正控制路徑的 update bit 只在 Update event 改變。」

11.2 回答「為什麼 CE/UE 只有一個 cycle」

「因為它們對應 TAP 的 Capture-DR 與 Update-DR 暫態 state，本來就只能停留一個 state interval；只有 Shift-DR 能透過 TMS=0 重複 N cycles。是否真的執行動作要看 register 的有效 edge，不只看 viewer 色塊。」

11.3 回答「為什麼 SIB 跳來跳去」

「波形中跳的是 shift stage，它和一般 scan bit 一樣會隨 TDI stream 移動；路徑 mux 由 update/select stage 控制。只要 update/select 在整段 Shift 穩定，並只在 UE 後改變，這個跳動就是正常資料移動。」

11.4 回答「TDO_EN 為什麼晚」

「我先確認它的極性與 pad connection。這個 net 接的是低有效 OEN，所以 Shift 期間為 0 才是 drive；最後一個 TDO bit 在 rising edge 已被 tester 取樣，下一個 falling edge OEN 拉高關閉輸出，因此視覺上會比 SE 結束晚半個 cycle。」

11.5 回答「一次能做幾個動作」

「對同一個 scan register，正常 transaction 的 Capture、Shift、Update event 是分開的；但 Shift 不是 input/output 二選一，而是每拍同時從 TDI 移入一 bit、從 TDO 移出舊的 tail bit。整條 active path 上的所有 SIB/TDR 也會在同一個 shift edge 一起動。」

12. Trace Sign-off Checklist

12.1 TAP / protocol

- Reset sequence 能把 TAP 帶到 Test-Logic-Reset，並確認 current IR/SIB reset state。
- IR scan 與 DR scan 清楚分段；current IR 只在 Update-IR 後改變。
- Capture-DR、Shift-DR、Update-DR sequence 合法；Pause/Exit 行為符合 TMS。
- CE/SE/UE 在有效 action edge 對應正確，沒有造成同一 register 的非預期同時動作。

12.2 SIB / topology

- 每個 SIB 的 shift stage 與 update/select stage 都已加入 waveform。
- active path 僅由 scan 開始前的 update/select state 決定。
- 每次 shift count 等於 active path 總 bit 數；包含所有 SIB、TDR、bypass/pipeline bits。
- nested SIB 的 parent/child 開啟順序與 reset/retention 行為已驗證。

12.3 Data / compare

- Capture source mapping 與 ICL 宣告一致。
- TDI chronological order、vector display order 與 register bit numbering 已分開記錄。
- queue model 逐 bit 產生 expected TDO；first/last shift edge 沒有 off-by-one。
- expected/mask/actual 在同一個 compare edge 對齊。

12.4 TDO / pad

- 已追出 raw TDO、retimed TDO、TDE、TDO_EN 與 pad_tdo 完整路徑。
- TDO_EN active polarity 由 pad library 或 assign 式證明，不靠名稱猜。
- 離開 Shift 後 TDO disable 的 falling-edge timing 合理；最後一個 care bit 仍有完整取樣 window。
- output disabled 時 pad_tdo 為 Z 或被系統上拉/下拉；internal tdo_data 不被誤當外部 drive。

12.5 Independent confidence

- 至少做過一個反轉 SIB open bit 的 negative control，checker 確實失敗。
- 至少做過一個 shift length ± 1 或 expected bit 反轉測試，證明 compare 不是永遠 masked。
- 若要正式 sign-off，保留 TAP edge table、active path list、serial trace 與 first mismatch screenshot。

附錄 A：最小 Reference Model 範本

```
class ScanModel:
    def __init__(self, icl):
        self.icl = icl
        self.sib_update = icl.reset_values()
        self.q = []

    def active_path(self):
        return self.icl.walk(self.sib_update)

    def capture(self, instrument_values):
        path = self.active_path()
        self.q = path.capture_bits(instrument_values, self.sib_update)

    def shift(self, tdi):
        expected_tdo = self.q[-1]
        self.q = [tdi] + self.q[:-1]
        return expected_tdo

    def update(self):
        slices = self.active_path().decode_queue(self.q)
        self.sib_update.update(slices.sib_values)
        return slices.instrument_updates
```

實作時再加入 TAP state model、IR selection、X/unknown propagation、compare mask、Pause-DR 與 multiple scan interfaces。最重要的是保留 `q[0]` near TDI / `q[-1]` near TDO 的固定定義，避免在不同工具格式間混淆。

附錄 B：Waveform Edge Log 範本

Edge	State before→after	TM S	Event	Path / N	TDI	Exp TDO	Act TDO	OE	Mask	Result

建議實際 trace 時另外建立一張「SIB state sheet」，每次 Update event 記錄所有 SIB update/select 值；Edge Log 只引用該 snapshot 的編號，避免單張表過寬。

附錄 C：標準、公開教材與工具資料

- [\[1\] IEEE Std 1687-2014 官方標準頁](#)
- [\[2\] IEEE Std 1687-2014 文件入口／摘要](#)
- [\[3\] IEEE P1687 Working Group, IJTAG Implementation Poster \(shift/update bits、SIB\)](#)
- [\[4\] Mentor Graphics, IEEE 1687 ITC 2017 Standards Tutorial \(network elements、over-shifting\)](#)
- [\[5\] Texas Instruments, IEEE Std 1149.1 JTAG Testability Primer \(TAP state machine\)](#)
- [\[6\] Intel, IEEE 1149.1 Boundary-Scan Testing \(TDO enable/tri-state timing\)](#)
- [\[7\] ASSET InterTech, IEEE P1687 IJTAG Tutorial \(gateway、SIB、ICL/PDL\)](#)
- [\[8\] Siemens, Tessent IJTAG Fact Sheet \(ICL extraction、network insertion 與 retargeting\)](#)
- [\[9\] Larsson et al., Accessing general IEEE Std. 1687 networks via functional interfaces \(separable interface\)](#)
- [\[10\] Texas Instruments, SN54/74ABT8245 JTAG timing example \(first/last shift edge\)](#)

註：本文件以標準語意與公開教材建立 trace 方法；實際 Tessent net name、clock edge 與 pad enable 極性，仍須以你手上的 generated RTL、ICL、pattern format 與 I/O library 為準。